

Batch 1 — Foundation

System Design Teacher Platform • Build summary with plain-language file explanations

What was delivered

Batch 1 builds the **skeleton** of the application. No user-facing features yet — the goal of this batch is to set up the architecture so that every future feature has a clean place to plug in. Think of it as building the foundation and electrical wiring of a house before any rooms are built.

The deliverable is a working **health-check endpoint** at `/health/deep` that pings every external dependency (database, cache, AI provider, etc.) and reports whether they are all reachable. If this endpoint returns green, it proves the entire architecture is wired correctly and we are ready to build features on top.

Total files generated	31
Lines of code (approx.)	~1,500
External dependencies	FastAPI, Pydantic, motor, redis, structlog, dependency-injector
Local services running	Backend + MongoDB + Mongo Express + Redis (4 containers)
Test coverage	3 integration tests covering <code>/health</code> and <code>/health/deep</code>

The big picture, in plain language

The architecture pattern: Ports & Adapters (Hexagonal). Imagine a power strip. The wall socket (*port*) defines a shape. Any plug (*adapter*) that matches the shape can connect to it — a lamp, a phone charger, a kettle. The strip doesn't know or care what's plugged in, only that it fits.

Our application works the same way. The business logic talks to abstract "sockets" (ports) like "the database" or "the AI provider." The actual plugs (adapters) are decided at startup based on environment: locally we plug in MongoDB and a stub AI; in production we'll plug in Cosmos DB and real OpenAI. The business logic never changes — we just swap plugs.

This is why Batch 1 has 7 "ports" (abstract socket definitions) and 7 matching local adapters (the plugs we use during development). Batch 5 will add the Azure adapters — those are different plugs that fit the same sockets.

The 7 capabilities the app needs

#	Capability (port)	What it does — in plain English
1	Auth Provider	Verifies that a user is who they claim to be (Microsoft login)
2	Database	Stores and retrieves users, questions, attempts, AI feedback
3	LLM Provider	Calls an AI model (like OpenAI) to generate or evaluate content
4	Cache	A fast, temporary memory to avoid repeating expensive operations
5	Rate Limiter	Counts how often something happens and blocks excess (saves cost)
6	Telemetry	Records logs, metrics, and errors so we can see what's happening
7	Secrets Provider	Fetches passwords and API keys safely, never logged

File-by-file breakdown (plain language)

Every file generated, what it does, and why it exists. Files are listed in the order they were generated and grouped by purpose.

Group 1: Project root & infrastructure (3 files)

Files at the top of the repo that set up tooling and the local development environment.

#1 `.gitignore`

What it is: A list of files Git should pretend don't exist (caches, virtual environments, secret files).

Why it exists: Stops accidentally committing junk or sensitive data to the repository. The first line of defense against leaking API keys and passwords.

#2 `README.md`

What it is: The first page anyone sees when they open the repository on GitHub. Project overview, tech stack, and a 4-command quickstart.

Why it exists: First impression for recruiters and contributors. Tells them what this project is, why it exists, and how to run it on their laptop in under 5 minutes.

#3 `docker-compose.yml`

What it is: A recipe for running the entire local stack with one command. Defines 4 services: backend, MongoDB, Mongo Express (a database browser), and Redis.

Why it exists: Removes the "works on my machine" problem. Anyone with Docker can run `docker compose up` and have an identical environment to yours.

Group 2: Architecture Decision Records (2 files)

Short documents explaining *why* we made a specific architectural choice. Future you (and interviewers) will thank you for these.

#4 `docs/adr/001-ports-and-adapters.md`

What it is: A 2-page document explaining the choice of Ports & Adapters architecture, alternatives we considered, and why we rejected them.

Why it exists: When someone asks "why did you use this pattern?" in an interview or code review, the answer is in this file. Shows mature engineering thinking.

#5 `docs/adr/006-dependency-injector.md`

What it is: Explains the choice of the `dependency-injector` Python library over manual wiring or FastAPI's built-in dependency system.

Why it exists: Same as above — a paper trail for why specific tools were chosen, in case we (or anyone else) need to revisit the decision later.

Group 3: Backend configuration (4 files)

Files that configure the Python project itself and how it gets packaged into a container.

#6 `backend/pyproject.toml`

What it is: The Python equivalent of `package.json` in Node.js. Lists every library the backend depends on and configures the linter, type checker, and test runner.

Why it exists: Single source of truth for dependencies. Anyone running `pip install -e .` gets the exact same versions, eliminating "it works on my machine" issues.

#7 `backend/Dockerfile`

What it is: Recipe for packaging the backend into a Docker container image. Two-stage build: install dependencies, then copy app code into a slim runtime image.

Why it exists: Same code runs identically in dev, on a CI server, and in Azure production. No surprises.

#8 `backend/.dockerignore`

What it is: Like `.gitignore` but for Docker. Tells Docker what NOT to copy into the container image.

Why it exists: Keeps images small (faster deploys) and prevents leaking secrets like `.env.local` into images.

#9 `backend/.env.example`

What it is: A template showing every environment variable the app reads. Real values go in `.env.local` (which is gitignored).

Why it exists: New contributors copy this to `.env.local`, fill in their values, and the app works. Documents every config knob in one place.

Group 4: Ports — the abstract "socket" definitions (7 files)

Each of these defines a contract: "anyone who wants to be the database/cache/AI provider must implement these methods." They contain NO actual logic — they are blueprints, not buildings.

#10 `app/core/ports/auth_provider.py`

What it is: Defines what an authentication provider must do: verify a user's login token and tell us who they are.

Why it exists: Lets us swap mock authentication (in development) for real Microsoft authentication (in production) without changing any business code.

#11 `app/core/ports/database.py`

What it is: Defines what a database must do, organized by collection (users, questions, attempts, cached feedback).

Why it exists: Lets us use local MongoDB during development and Azure Cosmos DB in production. Both speak the same protocol so the same code works for both.

#12 `app/core/ports/llm_provider.py`

What it is: Defines what an AI provider must do: generate text and generate structured JSON responses, while reporting how many tokens it used.

Why it exists: Lets us swap a stub AI (Batch 1, no real API calls) for real OpenAI (Batch 4) without touching any other code. Token counting is built in so we can monitor costs.

#13 `app/core/ports/cache.py`

What it is: Defines what a cache must do: get, set with optional expiry, delete.

Why it exists: Lets us use Redis locally and a Cosmos container in production. Caching saves dramatic amounts of money on AI calls — same question, same answer, no need to pay twice.

#14 `app/core/ports/rate_limiter.py`

What it is: Defines what a rate limiter must do: atomically count how many times something happens in a time window, and decide whether to allow or block.

Why it exists: Protects against runaway costs. If 1000 users hit the AI endpoint in an hour, the rate limiter stops the bill from exploding.

#15 `app/core/ports/telemetry.py`

What it is: Defines what a logging/metrics system must do: emit log events, record metrics, track exceptions.

Why it exists: Lets us use simple console logs locally and rich Application Insights dashboards in production. Same logging code works for both.

#16 `app/core/ports/secrets_provider.py`

What it is: Defines what a secrets fetcher must do: get a secret value by name.

Why it exists: Lets us read secrets from `.env` files locally and from Azure Key Vault in production. Includes a hard rule that secret values must NEVER be logged.

Group 5: Local adapters — the actual "plugins" we use (7 files)

Each of these is a concrete implementation of one of the ports above. These are the real working code that talks to MongoDB, Redis, etc. on your laptop.

#17 `app/adapters/local/mock_auth.py`

What it is: A fake login system that pretends every login token is valid and returns a hardcoded test user.

Why it exists: Lets us develop and test the entire app without setting up a real Microsoft tenant or app registration. The real one comes in Batch 5.

#18 `app/adapters/local/mongodb_database.py`

What it is: Real MongoDB client using the async motor driver. One class per collection (users, questions, attempts, feedback cache).

Why it exists: Connects to the MongoDB container started by `docker compose`. The EXACT same code will work against Azure Cosmos DB later — Cosmos speaks MongoDB's protocol.

#19 `app/adapters/local/stub_llm.py`

What it is: A fake AI that returns canned text instead of calling OpenAI. Counts roughly how many tokens would have been used.

Why it exists: Lets us develop and test without spending money on real API calls. The real OpenAI adapter arrives in Batch 4 when we have actual feedback prompts to send.

#20 `app/adapters/local/redis_cache.py`

What it is: Real Redis client (async) connected to the Redis container. Implements get/set/delete with TTL.

Why it exists: Fast, in-memory caching for development. In production this gets replaced with a Cosmos DB container that has TTL indexing — same interface, different storage.

#21 `app/adapters/local/memory_rate_limiter.py`

What it is: Counts requests in a Python dictionary in memory. Uses an asyncio lock so simultaneous requests don't double-count.

Why it exists: Good enough for local development (single process). Production needs a shared counter across workers, which Batch 5 adds via Cosmos DB.

#22 `app/adapters/local/console_telemetry.py`

What it is: Prints structured JSON logs to your terminal using the `structlog` library. Automatically redacts anything that looks like a password or token.

Why it exists: Easy to read and grep during development. Production uses Application Insights — same calling code, different destination.

#23 `app/adapters/local/env_secrets.py`

What it is: Reads secrets from environment variables. "openai-api-key" becomes `OPENAI_API_KEY` automatically.

Why it exists: Simple and standard for local dev. Production uses Azure Key Vault — same calling code, different storage.

Group 6: Wiring & service layer (3 files)

The glue that connects everything. Reads configuration, decides which adapter to use for each port, and provides services that use them.

#24 `app/config/settings.py`

What it is: Reads environment variables (and `.env.local`) into a typed Python object. Validates them at startup.

Why it exists: If you forget to set a required setting, the app fails to start with a clear error — instead of crashing later with a confusing message.

#25 `app/config/container.py`

What it is: The dependency-injection container. Says: "when business code asks for the database, give it the MongoDB adapter; when it asks for the AI, give it the stub."

Why it exists: Single place where ports are connected to adapters. To switch to Azure, we change this one file. No business logic ever touches it.

#26 `app/core/services/health_service.py`

What it is: A service that asks every port "are you healthy?" in parallel, then aggregates the answers into a report.

Why it exists: Powers the `/health/deep` endpoint. If any component is broken, this is what tells us — and tells deployment systems not to send traffic yet.

Group 7: API & entry point (2 files)

Where HTTP requests come in and the app starts up.

#27 `app/api/routes/health.py`

What it is: Defines two HTTP endpoints: `/health` (quick "am I alive?") and `/health/deep` (full dependency check).

Why it exists: `/health` is hit by the load balancer to decide if the container is alive. `/health/deep` is for humans to verify everything works.

#28 `app/main.py`

What it is: The FastAPI application factory. Builds the dependency container, sets up CORS, registers routes, and handles startup/shutdown cleanup.

Why it exists: This is what uvicorn runs. The single entry point that ties the whole backend together.

Group 8: Tests (1 file)

#29 `tests/test_health_smoke.py`

What it is: Three tests: (1) shallow health returns 200, (2) deep health returns 200 with all 7 components green, (3) deep health correctly returns 503 when something is broken.

Why it exists: The gating proof that Batch 1 works. If these pass, the architecture is wired correctly and we are ready for Batch 2.

Group 9: Plumbing (12 files)

#30 (×11) Various `__init__.py` files

What it is: Empty files that mark a directory as a Python package so imports work.

Why it exists: Pure plumbing. Python requires these (at least, the way we're set up). They contain nothing.

#31 `frontend/.gitkeep`

What it is: Empty placeholder file in the frontend directory.

Why it exists: Git won't track empty directories. This file makes sure the empty `frontend/` directory is tracked until Batch 2 fills it with React code.

How the pieces fit together

Here's what happens when someone hits `GET /health/deep`:

1. `uvicorn` receives the HTTP request and routes it to `FastAPI`.
2. `FastAPI` matches the URL to the handler in `api/routes/health.py`.
3. The handler asks the DI container for a `HealthService` instance.
4. The DI container hands back a `HealthService` with all 7 ports already injected (each pointing to its local adapter).
5. `HealthService.check_deep()` runs all 7 health checks *at the same time* using `asyncio.gather`.
6. Each adapter does its check: MongoDB pings the DB, Redis pings the cache, the stub AI returns OK instantly, etc.
7. Results are collected into a `DeepHealthReport`.
8. If everything is OK, returns 200. If anything failed, returns 503 with details.
9. The result is also logged via the telemetry port, so production dashboards can graph health over time.

How to verify Batch 1 works

Step 1 — Set up environment

```
cp backend/.env.example backend/.env.local
```

No need to edit it — the OpenAI placeholder is fine because Batch 1's AI is stubbed.

Step 2 — Bring up the local stack

```
docker compose up --build
```

Four services start: backend, MongoDB, Mongo Express, Redis. Watch for "Uvicorn running on http://0.0.0.0:8000" in the logs.

Step 3 — Hit the endpoints

```
curl http://localhost:8000/health
curl http://localhost:8000/health/deep | jq
```

First should return `{"status": "ok"}`. Second should return a JSON body with all 7 components reporting `"status": "ok"`.

Step 4 — Run the tests

```
docker compose exec backend pytest tests/ -v
```

Expected: 3 tests pass.

What's next: Batch 2 — Auth & User

Now that the foundation is solid, Batch 2 adds the first user-facing piece:

- Frontend scaffold: React + Vite + TypeScript
- Microsoft login flow via MSAL.js (still mocked locally — real Microsoft tenant in Batch 5)
- Backend issues its own JWT after successful login
- User domain entity + repository methods (insert, look-up by Microsoft ID)
- `frontend/AGENT.md` with React conventions
- ADR-003 documenting why direct OpenAI rather than Azure OpenAI

To begin Batch 2, say: "start Batch 2".